Laboratory 2 – GPIO - Inputs

1 Overview and Goals

In the previous laboratory the Texas Instruments (TI) MSP430F5438 Experimenter microcontroller board was introduced along with the MSP+430 development tool Code Composer Studio (CCS). The classic example of a first embedded program, which involved setting up the ports connected to the LEDs as outputs and blinking the LED using a software delay. Within this laboratory we will extend on these ideas.

There are several specific objectives in this lab:

1. Investigate in further detail the operation of the GPIO ports on the MSP430F5438
2. Understand the function of pull up and pull down resistors.
3. Interface with the push button switches via GPIO.
4. Use control logic to manipulate outputs.

2 Introduction

As we have seen in the previous laboratory the MCU senses the world from its input pins and communicates with the world via output pins. To give the designer flexibility when designing electronic circuits most microcontrollers have I/O systems that can be configured as either inputs or outputs. These I/O systems are often pin multiplexed with special functional units which perform more advanced operations such as analog to digital conversion (ADC) or serial communications. Within this laboratory we will focus on the second half of I/O, setting up the system for input.

2.1 Useful Documentation

To help new developers get up to speed with the rich set of functionality that is available with the MSP430 there exists extensive documentation, relating to both the MCU and the Experimenter board. The following wiki is populated by TI and is a good place to begin to find documentation and software examples.

http://processors.wiki.ti.com/index.php/MSP-EXP430F5438_Experimenter_Board

The following documents (pdfs) will be required for this and any subsequent laboratories, they are available at the following website and copies are available on GCU Learn.

http://www.ti.com/tool/msp-exp430f5438

- MSPF5438 Experimenter User Guide (Rev G.)

http://www.ti.com/product/msp430f5438a

- MSP430F543xA, MSP430F541xA Mixed Signal Microcontroller (Rev. B)

Datasheet for the MSP430F5438
http://www.ti.com/lit/ds/symlink/msp430f5438.pdf

3 Digital Output using General Purpose Input Output (GPIO)

In the previous laboratory, we have setup Port 1.0 as an output port by setting the data direction register to configure the port as an output.

```
// Setup P1.0 and an output port
P1DIR = P1DIR | BIT0;
```

We then could output to the port that is connected to the LED as follows:

```
// Toggle the LED
P1OUT ^= BIT0;
```

In this laboratory we are going to follow the same logic to take input from the push button switches, and use this initially to control the LEDs.

3.1 Microcontroller peripherals - Inputs

The simplest input available to communicate with a microcontroller is a switch. There are two push button switches available to us on the MSP430F5438 Experimenter board. As we wish to take input from the push button switches, we must first find out what pins these are attached to on the board. (You should have completed this in the previous laboratory). We find this by referring to the circtuit diagram for the board. The appropriate section of the circtuit diagram is shown in Figure 1.

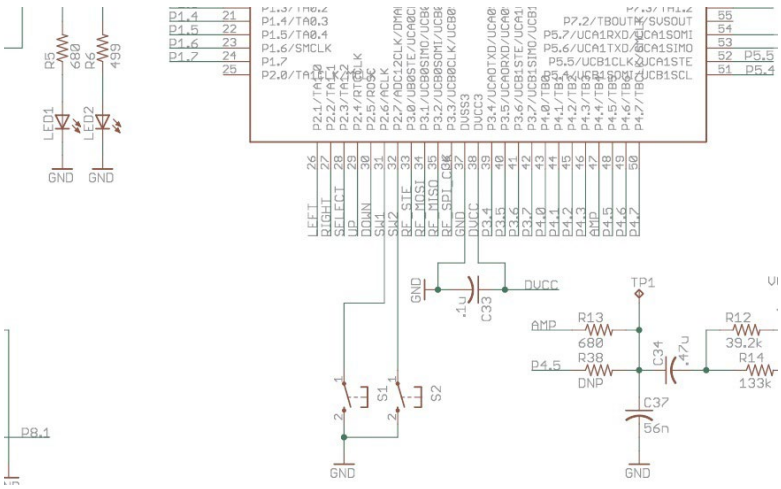

Figure 1. Subsection of the circuit diagram MSP430F5438 Experimenter board

From the above diagram we can see that push button switch 1 (s1) is connected to port 2.6 and push button switch 2 (s2) is connected to port 2.7.

In order to develop software to read input from the swtiches to control the LEDs, you'll need to know a bit about how port are setup as inputs. This is essentially the reverse of the configuration we used in the previous laboratory. We saw previously that each pin's input/output status (i.e. the direction that the pin operates in input or output) can be individually set by manipulating 'registers' in the microcontroller, each register is a group of 8 bits within the microcontroller that may or may not directly affect pins. Once this is set we can then read or write to the pin. Thus to setup the port as an input we must set the appropriate bit in the data direction register.

Lab Question 1. Write the code to configure both port 2.6 and 2.7 as inputs? (This can be done in one line)

```
P2DIR=P2DIR&~ BIT6;   % Configure Direction register for 2.6
P2SEL=P2SEL&~ BIT6;
P2DIR=P2DIR&~ BIT7;
P2SEL=P2SEL&~ BIT7;
```

3.2 Pin Multiplexing

Looking at the schematic representation of the MSP430 (MSP430F5438 in Figure 1) you will notice that each pin has a long name, with several designations separated by a slash. Because microcontrollers have a limited number of pins, the manufacturer has to multiplex the pins among the internal modules. That is, each pin can only do one things at a time and you have to select what that function will be (upon startup there are some defaults which you will likely override with your code). Multiplexing can become a problem if you're in short supply of I/O pins and you're using others for specialized functions. In your design stage you should take all of this into account and plan which pins will be used (and with what functionality).

Most pins on the MSP430 devices include GPIO functionality. In their naming, this is indicated by PX.Y, where the X represents the port number and the Y the pin number in that port. MSP430 devices have several different ports, depending on the actual device, and these ports each have 8 pins (making byte access easy). For example, Port 1 is represented by pins P1.0 P1.1, P1.2, P1.3,..., P1.7.

Lab Question 2. From your notes from the previous laboratory, state which register within port 1.x is utilized to select between digitial I/O and a special function?

Lab Question 3. Write the code to configure port 2.6 and 2.7 to utilize digital I/O and not a special function?

3.2 Pullup Resistors

If at this point we were to configure the pins on port 2.6 and 2.7 as inputs, we can see from the circuit diagram that with the switch open and nothing connected to the pins, then the value is said to be floating. This means the MCU might have a difficult time reading the state or voltage on the input pin,

this is a situation that we wish to avoid. To prevent this a pull-up or pull-down resistor will hold the pin to either a high or low state, while using a low amount of current. This is shown in the following figure.

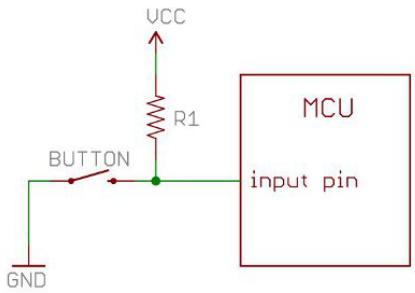

Figure 2. Pullup resistor, ensures the value seen at the pin is high when the swich is not pressed.

With a pull-up resistor, the input pin will read a high state when the button is open. When the button closes, it connects the input pin directly to ground, thus reading a low state.

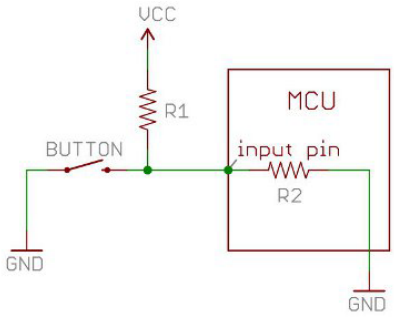

Figure 3. Pullup resistor, and internal resistance

The value of the pull-up resistor needs to be chosen to satisfy two conditions:

1. When the button is pressed, the input pin is pulled low. The value of the resistor controls how much current you want to flow from VCC through the resistor R1, through the button, and then to ground.
2. When the button is not pressed, the input pin is pulled high. The value of the pull-up resistor controls the voltage on the input pin.

Lab Question 4. The above circuit diagram shows the pull up resistor configuration which ensures the MCU sees the pin in a high state when the switch is not pressed. Update the above diagram to show the pull down resistor configuration where the MCU will see the pin to be in a low state ?

Many microcontrollers include internal pull-up or even pull-down, this is software configurable, which free the designer from having to include them. In this case, simply connect the button to either VCC and GND and it should work. This is the case with the MSP430F5438, we have the option to configure the pin with either an internal pull up or pulldown resistor. As the switch S1 is connected to ground (GND), we would require a pullup resistor in this case. We can think of the internal circuit as follows:

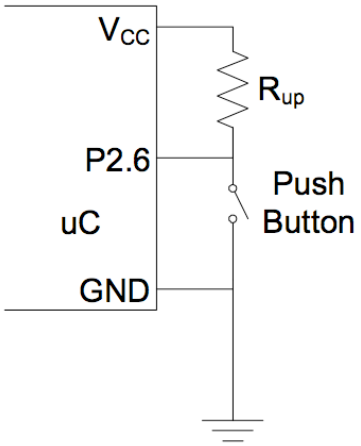

Figure 4. Pullup resistor internal circuit

Each port is assigned several 8 bit registers that control them and provide information about their current status. Each register is designated PxYYY, where the lowercase x represents the port number and YYY represent the name of the functionality of the register. At this stage, you need to know about the following registers.

PxSEL	Set function of each pin in Px. 0=basic digital I/O 1=special function
PxDIR	Set data flow direction of each pin in Px. 0=pin is input 1=pin is output
PxOUT	Set voltage on each output pin in Px. 0=pin low (GND) 1=pin high (Vcc)
PxIN	Read voltage on each pin in Px. 0=pin low (GND) 1=pin high (Vcc)
PxREN	Enable or disable R_{up} on each pin in Px. 0=R_{up} disabled 1=R_{up} enabled

To setup the ports to operate as we wish them to we require to manipulate the above registers. The above registers are 8 bits in size, yet we only wish to manipulate individual bits as we saw in the previous laboratory. To do this we require to be able manipulate individual bits within a register in software. To do this we can utilize some convienient predefined values BIT0 through BIT7. These are defined in the include file msp4305438.h as seen previously.

```
#define BIT0                    (0x0001)
#define BIT1                    (0x0002)
#define BIT2                    (0x0004)
#define BIT3                    (0x0008)
#define BIT4                    (0x0010)
#define BIT5                    (0x0020)
#define BIT6                    (0x0040)
#define BIT7                    (0x0080)
```

PxREN - This register controls the internal pullup and pulldown resistors of each pin. These can be important when using switches. Instead of using an external pullup resistor, we can simply configure everything internally, saving space. The corresponding bit in the PxOUT register selects if the pin is pulled up or pulled down.

In PxREN, a 0 in a bit means that the pullup/pulldown resistor is disabled and a 1 in any bit signifies that the pullup/pulldown resistor is enabled. Once you select any pin to have the pullup/pulldown resistor active(by putting a 1 in the corresponding bit), you must use PxOUT to select whether the pin is pulled down (a 0 in the corresponding bit) or pulled up (a 1 in the corresponding bit).

IMPORTANT

Thus previously we have seen the following register used if a pin was setup for output.

PxOUT	Set voltage on each output pin in Px. 0=pin low (GND) 1=pin high (Vcc)

If a pin is setup for input and pullup PxREN has been set to enable the pullup/pulldown resistor, then this changes, and we must set PxOUT as follows:

PxOUT	Set pullup direction. 0=pulldown 1=pullup

The pullup/pulldown is useful in many situations, and one of the most common one is switches connected to the MSP430.

Lab Question 5. Within this laboratory we wish to control the operation of the LEDs LED1 and LED2 using the push button switches s1 and s2, from the table above, list the registers that we need to set to achieve this for the appropriate port, write the code in the table below, use N/A if not applicable.

	P1.0	P1.1	P2.6	P2.7
PxSEL				
PxDIR				
PxOUT				
PxIN				
PxREN				

(20 marks, one for each box)

Following the same procedure that was utilized in the first laboratory.

1. Create a new project, named testInput.
2. Add a main.c file to the project.
3. Enter the following code into the main function.

4. Compile the code.

5. Download the code onto the board.

6. Run the code.

```
//*******************************************************************************
//   Description: Setup the system to receive input from switch 1 (SW1), use this
//                to control the LED (LED1)
//   Author:
//   Data:
//*******************************************************************************

#include "msp430x54x.h"

void main(void)
{
    // Stop WDT
    WDTCTL = WDTPW + WDTHOLD;

    // Setup P1.0 as an output port
    P1DIR = P1DIR | BIT0;
    P1SEL = P1SEL & ~BIT0;

    // Setup P2.6 as an input port
    P2DIR = P2DIR & ~BIT6;
    P2SEL = P2SEL & ~BIT6;

    // Enable the pull up resistor and set as a pullup
    P2REN = P2REN | BIT6; // Pull up resistor enable
    P2OUT = P2OUT | BIT6; // Pull up resistor

    // Create main loop
    while(1)
    {
        // If S1 is not pressed
        if(P2IN & BIT6)
        {
                // Turn on the LED
                P1OUT |= BIT1;
        }
        else
        {
                // Turn off the LED
                P1OUT &= ~BIT1;
        }
    }

    return;
}
```

Lab Question 6. Explain the operation of the above code, detailing every line. Does it operate as you expect?

```
1. void main(void)
2. {
    // Stop WDT
3.     WDTCTL = WDTPW + WDTHOLD;

    // Setup P1.1 as an output port
4.     P1DIR = P1DIR | BIT0;
5.     P1SEL = P1SEL & ~BIT0;

    // Setup P2.6 as an input port
6.    P2DIR = P2DIR & ~BIT6;
7.    P2SEL = P2SEL & ~BIT6;
```

```
    // Enable the pull up resistor and set as a pullup
8.    P2REN = P2REN | BIT6;
9.    P2OUT = P2OUT | BIT6;

    // Create main loop
10.    while(1)
11.    {
       // If S1 is pressed
12.    if(P2IN & BIT6)
13.    {
               // Turn on the LED
14.            P1OUT |= BIT0;
15.    }
16.    else
17.    {
               // Turn off the LED
18.            P1OUT &= ~BIT0;
19.    }
20.    }

21.    return;
22.}
23.
```

1 – beginning of the main function, this is where the program enters
2 – Opening brace for main function
3 – stops the watchdog timer, allowing us to create infinite loops
4 – sets port 1.0 as an output by setting bit 0 of the register P1DIR to 1
5 – sets port 1.0 for general purpose I/O operations by clearing bit 0 of register P1SEL
6 - sets port 2.6 as an output port by clearing bit 6 of register p2DIR
7 – sets port 2.6 for general purpose I/O operations by by clearing bit 6 of register p2SEL
8 – enables the pullup resistor on port 2.6 by setting bit 6 of register P2REN to 1
9 – pulls the pullup resistor high by setting bit 6 of P2OUT to 1
10 – start of the main loop, this allows us to keep program control within the loop
11 – opening brace for while loop
12 – checks to see if the value contained in bit 6 of register P2IN is 1(checks if the button is not pressed)
13 – opening brace for if statement
14 – if the value in bit 6 of register P2IN is 1, then turn on the LED by setting bit 0 of register P1OUT to 1
15 – closing brace for if statement
16 – otherwise, if the switch 1 has not been pressed, i.e. the value of bit 6 in register is 0
17 – opening brace for else statement
18 – switch off LED1 byclearing the value contained within bit 0 of register P1OUT with 1, by ANDing the bit with the inverted bit mask
19 – else statement closing brace
20 – while loop closing brace
21 – return statement for main function – not needed
22 – closing brace for main function
23 – blank line – should be included at the end of every main.c file
(10 marks)

Lab Question 7. Change the above code so that LED 1 is only on when switch S1 is pressed.

Lab Question 8. Modify the code above to take input from switch 2 (s2), and turn on LED 2, only when s2 is pressed.

Lab Question 9. Modify the code above to take input from switchs 1&2 (s1 & s2), and turn on both LEDs 1&2, only when both s1 AND s2 are NOT pressed.

Lab Question 10. Modify the code above and utilizing the delay function that you used last week, create and and call a function that flashes both LEDs n times, if switch 1 is pressed. The value n should be used as a paramenter to the function.

)

3.4 Next laboratory – Interrupts, Low Power and Watchdog Timer

In this laboratory we read input from the switches, and utilized this to control the LEDs. Yet to do this we had to continually poll the switches in software to check for a change on the input. This is quite inefficient, especially if the change on the input occurs infrequently. Next week we will introduce interrupts, which allow us to respond to input only when it occurs, leaving the processor to perform other operations without having to poll for input.

Lab Question 11. What are interrupts and why are they useful in embedded software?

So far all of the software we have written has contained the following line of code

```
    // Stop WDT
    WDTCTL = WDTPW + WDTHOLD;
```

Which from the comment we know is used to stop the watchdog timer.

Lab Question 12. What is the function of a watchdog timer and why is it often used?

Lab Question 13. The MSP430 comes with a number of available low power modes, detail them?